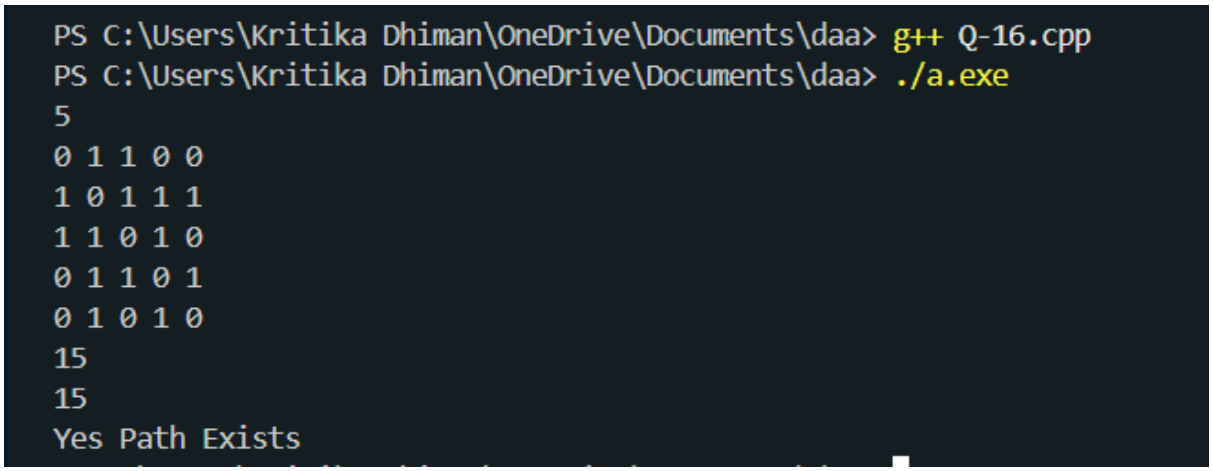
WEEK 6:

/*I. Given a (directed/undirected) graph, design an algorithm and implement it using a program to
find if a path exists between two given vertices or not. (Hint: use DFS)*/

```
#include <iostream>
#include <vector>
using namespace std;

bool pathExists(int current, int destination, vector<vector<int>> &graph, vector<bool>
&visited, int vertices)
{
   if(current == destination)
      return true;

   visited[current] = true;

   for(int i = 1; i <= vertices; i++)   // 1-based indexing
   {
      if(graph[current][i] == 1 && !visited[i])
      {
         if(pathExists(i, destination, graph, visited, vertices))
            return true;
      }
   }

   return false;
}

int main()
{
   int vertices;
   cin >> vertices;
vector<vector<int>> graph(vertices + 1, vector<int>(vertices + 1));
for(int i = 1; i <= vertices; i++)
   {
      for(int j = 1; j <= vertices; j++)
      {
         cin >> graph[i][j];
      }
   } int source, destination;
   cin >> source >> destination;

   vector<bool> visited(vertices + 1, false);
if(pathExists(source, destination, graph, visited, vertices))
      cout << "Yes Path Exists";
   else
      cout << "No Such Path Exists";

   return 0;}
```

OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-16.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 1 1 0 0
1 0 1 1 1
1 1 0 1 0
0 1 1 0 1
0 1 0 1 0
15
15
Yes Path Exists

/*II. Given a graph, design an algorithm and implement it using a program to find if a graph is bipartite or not. (Hint: use BFS)
USING ADJANCENCY LIST */

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

bool isBipartite(vector<vector<int>> &graph, int vertices)
{
   vector<int> color(vertices, -1);  // -1 = not colored

   for(int start = 0; start < vertices; start++)
   {
      if(color[start] == -1)
      {
         queue<int> q;
         q.push(start);
         color[start] = 0;

         while(!q.empty())
         {
            int node = q.front();
            q.pop();

            for(int i = 0; i < vertices; i++)
            {
               if(graph[node][i] == 1)   // edge exists
               {
                  if(color[i] == -1)
                  {
                     color[i] = 1 - color[node];
                     q.push(i);
                  }
                  else if(color[i] == color[node])
                  {
                     return false;   // same color → not bipartite
                  }
               }
            }
         }
      }
   }

   return true;
}

int main()
{
```

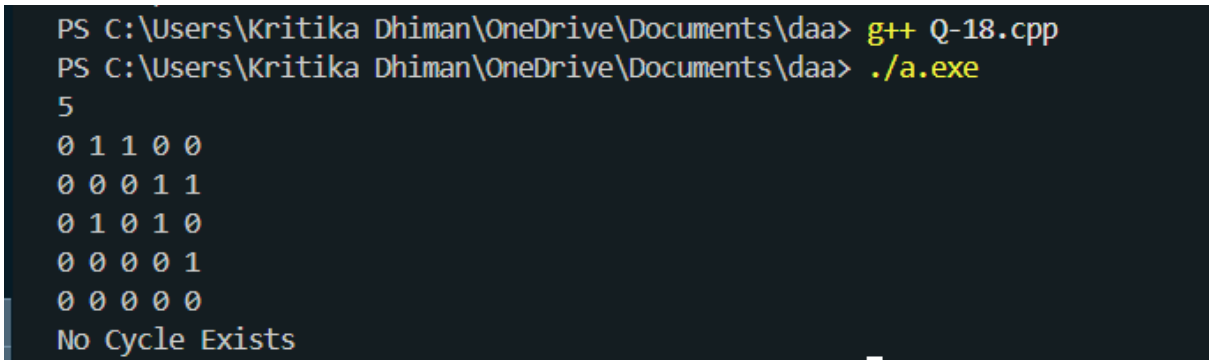
```
    int vertices;
    cin >> vertices;

    vector<vector<int>> graph(vertices, vector<int>(vertices));

    // input adjacency matrix
    for(int i = 0; i < vertices; i++)
    {
        for(int j = 0; j < vertices; j++)
        {
            cin >> graph[i][j];
        }
    }

    if(isBipartite(graph, vertices))
        cout << "Yes Bipartite";
    else
        cout << "Not Bipartite";

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-17.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 1 1 0 0
1 0 1 1 1
1 1 0 1 0
0 1 1 0 1
0 1 0 1 0
Not Bipartite
```

```
/*III.Given a directed graph, design an algorithm and implement it using a program to find whether
cycle exists in the graph or not. USING ADJACENCY LIST */
#include<iostream>
#include<vector>
using namespace std;
bool dfs(int node,vector<vector<int>>&graph,vector<bool>&visited,vector<bool>&rec,int v){
visited[node]=true;
rec[node]=true;
for(int i=0;i<v;i++){
if(graph[node][i]==1){
if(!visited[i]&&dfs(i,graph,visited,rec,v))return true;
else if(rec[i])return true;
}
}
rec[node]=false;
return false;
}
int main(){
int v;
cin>>v;
vector<vector<int>>graph(v,vector<int>(v));
for(int i=0;i<v;i++)
for(int j=0;j<v;j++)
cin>>graph[i][j];
vector<bool>visited(v,false),rec(v,false);
for(int i=0;i<v;i++){
if(!visited[i]){
if(dfs(i,graph,visited,rec,v)){
cout<<"Yes Cycle Exists";
return 0;
}
}
}
cout<<"No Cycle Exists";
return 0;
}
```

OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-18.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 1 1 0 0
0 0 0 1 1
0 1 0 1 0
0 0 0 0 1
0 0 0 0 0
No Cycle Exists

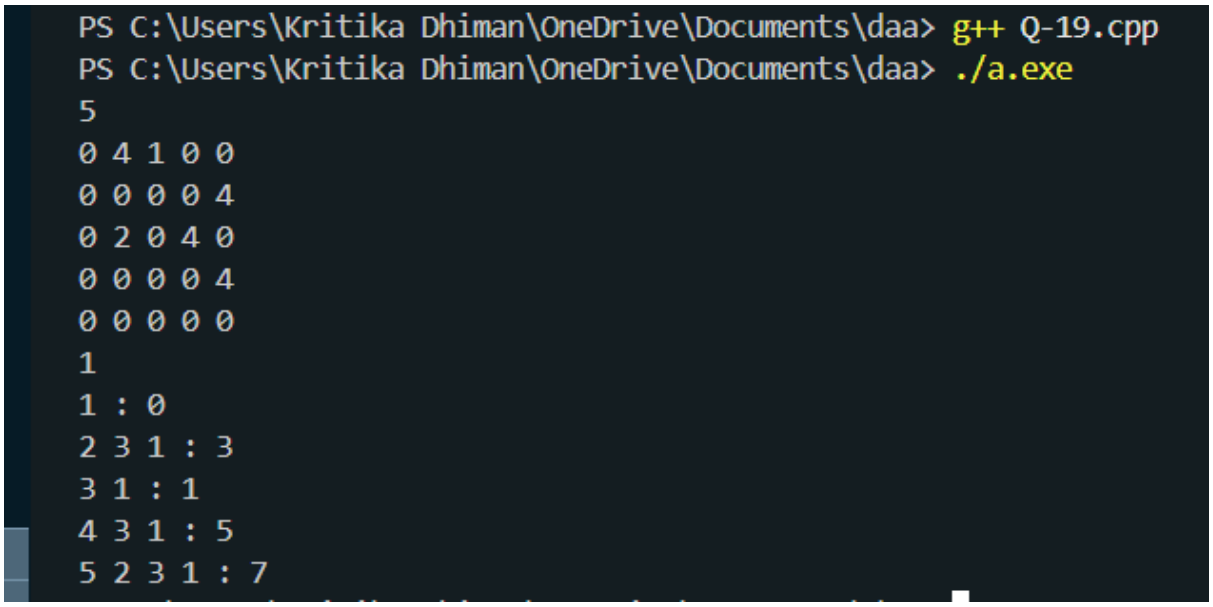
WEEK 7:

/*I.After end term examination, Akshay wants to party with his friends. All his friends are living as
paying guest and it has been decided to first gather at Akshay's house and then move towards party location. The problem is that no one knows the exact address of his house in the city. Akshay as a computer science wizard knows how to apply his theory subjects in his real life and
came up with an amazing idea to help his friends. He draws a graph by looking in to location of
his house and his friends' location (as a node in the graph) on a map. He wishes to find out shortest distance and path covering that distance from each of his friend's location to his house
and then whatsapp them this path so that they can reach his house in minimum time. Akshay has
developed the program that implements Dijkstra's algorithm but not sure about correctness of results. Can you also implement the same algorithm and verify the correctness of Akshay's results? (Hint: Print shortest path and distance from friends' location to Akshay's house)*/

```
#include<iostream>
#include<vector>
#include<climits>
using namespace std;

int main() {
   int v;
   cin >> v;

   vector<vector<int>> graph(v, vector<int>(v));

   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         cin >> graph[i][j];
      }
   }

   int src;
   cin >> src;
   src--;

   vector<int> dist(v, INT_MAX), parent(v, -1);
   vector<bool> visited(v, false);

   dist[src] = 0;

   for(int i = 0; i < v - 1; i++) {
      int u = -1, minDist = INT_MAX;

      for(int j = 0; j < v; j++) {
         if(!visited[j] && dist[j] < minDist) {
            minDist = dist[j];
```

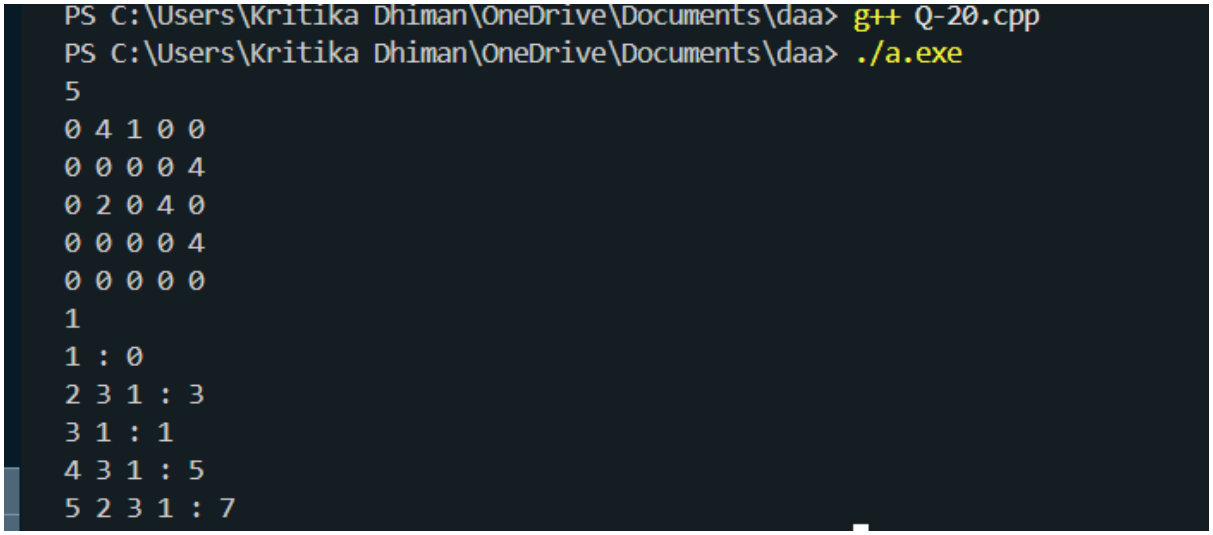
```
            u = j;
        }
    }

    if(u == -1) break;

    visited[u] = true;

    for(int j = 0; j < v; j++) {
        if(graph[u][j] > 0 && !visited[j] && dist[u] + graph[u][j] < dist[j]) {
            dist[j] = dist[u] + graph[u][j];
            parent[j] = u;
        }
    }
}

for(int i = 0; i < v; i++) {
    int curr = i;
    vector<int> path;

    while(curr != -1) {
        path.push_back(curr + 1);
        curr = parent[curr];
    }

    for(int j = 0; j < path.size(); j++) {
        cout << path[j];
        if(j < path.size() - 1) cout << " ";
    }

    cout << " : ";

    if(dist[i] == INT_MAX) cout << "INF";
    else cout << dist[i];

    cout << endl;
}

return 0;
}
```

OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-19.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 4 1 0 0
0 0 0 0 4
0 2 0 4 0
0 0 0 0 4
0 0 0 0 0
1
1 : 0
2 3 1 : 3
3 1 : 1
4 3 1 : 5
5 2 3 1 : 7

```
/*II.Design an algorithm and implement it using a program to solve previous question's
problem using Bellman- Ford's shortest path algorithm.*/
#include<iostream>
#include<vector>
#include<climits>
using namespace std;
int main() {
   int v;
   cin >> v;

   vector<vector<int>> graph(v, vector<int>(v));

   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         cin >> graph[i][j];
      }
   }int src;
   cin >> src;
   src--;

   vector<int> dist(v, INT_MAX), parent(v, -1);
   dist[src] = 0;

   for(int k = 0; k < v - 1; k++) {
      for(int i = 0; i < v; i++) {
         for(int j = 0; j < v; j++) {
            if(graph[i][j] != 0 && dist[i] != INT_MAX && dist[i] + graph[i][j] < dist[j]) {
               dist[j] = dist[i] + graph[i][j];
               parent[j] = i;
            }
         }
      }
   }for(int i = 0; i < v; i++) {
      int curr = i;
      vector<int> path;
      while(curr != -1) {
         path.push_back(curr + 1);
         curr = parent[curr];
      } for(int j = 0; j < path.size(); j++) {
         cout << path[j];
         if(j < path.size() - 1) cout << " ";
      }cout << " : ";

      if(dist[i] == INT_MAX) cout << "INF";
      else cout << dist[i];

      cout << endl;
   }return 0;
}
```

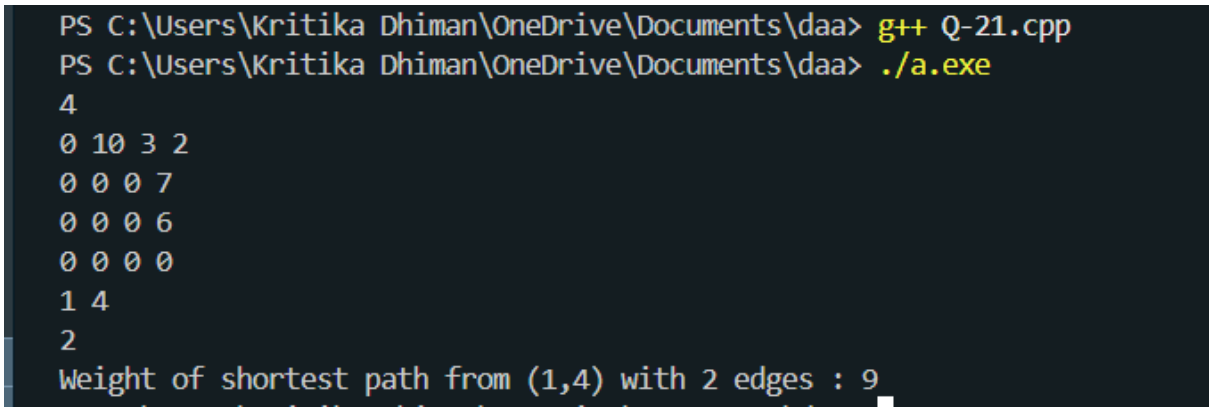
OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-20.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 4 1 0 0
0 0 0 0 4
0 2 0 4 0
0 0 0 0 4
0 0 0 0 0
1
1 : 0
2 3 1 : 3
3 1 : 1
4 3 1 : 5
5 2 3 1 : 7

/*III.Given a directed graph with two vertices (source and destination). Design an algorithm and
implement it using a program to find the weight of the shortest path from source to destination
with exactly k edges on the path*/

```
#include<iostream>
#include<vector>
#include<climits>
using namespace std;

int main() {
   int v;
   cin >> v;

   vector<vector<int>> graph(v, vector<int>(v));

   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         cin >> graph[i][j];
      }
   }

   int src, dest, k;
   cin >> src >> dest;
   cin >> k;

   src--;
   dest--;

   const int INF = 1e9;

   vector<vector<int>> dp(k + 1, vector<int>(v, INF));
   dp[0][src] = 0;

   for(int e = 1; e <= k; e++) {
      for(int i = 0; i < v; i++) {
         for(int j = 0; j < v; j++) {
            if(graph[j][i] != 0 && dp[e - 1][j] != INF) {
               dp[e][i] = min(dp[e][i], dp[e - 1][j] + graph[j][i]);
            }
         }
      }
   } if(dp[k][dest] == INF)
      cout << "no path of length k is available";
   else
      cout << "Weight of shortest path from (" << src + 1 << "," << dest + 1 << ") with " << k
<< " edges : " << dp[k][dest];

   return 0; }
```

OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-21.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
4
0 10 3 2
0 0 0 7
0 0 0 6
0 0 0 0
1 4
2
Weight of shortest path from (1,4) with 2 edges : 9

WEEK 8:

Assume that a project of road construction to connect some cities is given to your friend. Map of these cities and roads which will connect them (after construction) is provided to him in the form of a graph. Certain amount of rupees is associated with construction of each road. Your friend has to calculate the minimum budget required for this project. The budget should be designed in such a way that the cost of connecting the cities should be minimum and number of roads required to connect all the cities should be minimum (if there are N cities then only N-1 roads need to be constructed). He asks you for help. Now, you have to help your friend by designing an algorithm which will find minimum cost required to connect these cities. (use Prim's algorithm).

```
#include <stdio.h>
#include <limits.h>

int minKey(int key[], int visited[], int n) {
   int min = INT_MAX, index = -1;

   for (int i = 0; i < n; i++) {
      if (visited[i] == 0 && key[i] < min) {
         min = key[i];
         index = i;
      }
   }
   return index;
}

int primMST(int graph[100][100], int n) {
   int key[100], visited[100];
   int totalCost = 0;

   for (int i = 0; i < n; i++) {
      key[i] = INT_MAX;
      visited[i] = 0;
   }

   key[0] = 0;  // start from vertex 0

   for (int count = 0; count < n; count++) {
      int u = minKey(key, visited, n);
      visited[u] = 1;
      totalCost += key[u];

      for (int v = 0; v < n; v++) {
         if (graph[u][v] != 0 && visited[v] == 0 && graph[u][v] < key[v]) {
            key[v] = graph[u][v];
         }
      }
   }
```

```
    return totalCost;
}

int main() {
    int n;
    int graph[100][100];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int result = primMST(graph, n);
    printf("Minimum Spanning Weight: %d", result);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-22.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
7
0 0 7 5 0 0 0
0 0 8 5 0 0 0
7 8 0 9 7 0 0
5 0 9 0 15 6 0
0 5 7 15 0 8 9
0 0 0 6 8 0 11
0 0 0 0 9 11 0
Minimum Spanning Weight: 39
```

/*II. Implement the previous problem using Kruskal's algorithm.
Input Format:
The first line of input takes number of vertices in the graph.
Input will be the graph in the form of adjacency matrix or adjacency list.
Output Format:
Output will be minimum spanning weight*/

```
#include <stdio.h>

int find(int parent[], int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int parent[], int x, int y) {
    int rootX = find(parent, x);
    int rootY = find(parent, y);
    parent[rootX] = rootY;
}

int main() {
    int n;
    int graph[100][100];

    scanf("%d", &n);

    // input matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int edge_u[1000], edge_v[1000], edge_w[1000];
    int edgeCount = 0;

    // convert matrix to edge list
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (graph[i][j] != 0) {
                edge_u[edgeCount] = i;
                edge_v[edgeCount] = j;
                edge_w[edgeCount] = graph[i][j];
                edgeCount++;
            }
        }
    }

    // sort edges (simple bubble sort)
```

```
    for (int i = 0; i < edgeCount - 1; i++) {
        for (int j = 0; j < edgeCount - i - 1; j++) {
            if (edge_w[j] > edge_w[j + 1]) {
                int temp;

                temp = edge_w[j];
                edge_w[j] = edge_w[j + 1];
                edge_w[j + 1] = temp;

                temp = edge_u[j];
                edge_u[j] = edge_u[j + 1];
                edge_u[j + 1] = temp;

                temp = edge_v[j];
                edge_v[j] = edge_v[j + 1];
                edge_v[j + 1] = temp;
            }
        }
    }

    int parent[100];
    for (int i = 0; i < n; i++)
        parent[i] = i;

    int totalCost = 0;
    int edgesUsed = 0;

    for (int i = 0; i < edgeCount && edgesUsed < n - 1; i++) {
        int u = edge_u[i];
        int v = edge_v[i];

        int setU = find(parent, u);
        int setV = find(parent, v);

        if (setU != setV) {
            totalCost += edge_w[i];
            unionSet(parent, setU, setV);
            edgesUsed++;
        }
    }

    printf("Minimum Spanning Weight: %d", totalCost);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-23.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
7
0 0 7 5 0 0 0
0 0 8 5 0 0 0
7 8 0 9 7 0 0
5 0 9 0 15 6 0
0 5 7 15 0 8 9
0 0 0 6 8 0 11
0 0 0 0 9 11 0
Minimum Spanning Weight: 39
```

/*III.Assume that same road construction project is given to another person. The amount he will earn
from this project is directly proportional to the budget of the project. This person is greedy, so he
decided to maximize the budget by constructing those roads who have highest construction cost.
Design an algorithm and implement it using a program to find the maximum budget required for
the project.
Input Format:
The first line of input takes number of vertices in the graph.
Input will be the graph in the form of adjacency matrix or adjacency list.
Output Format:
Out will be maximum spanning weight.*/

```
#include <stdio.h>
#include <limits.h>

int maxKey(int key[], int visited[], int n) {
   int max = INT_MIN, index = -1;

   for (int i = 0; i < n; i++) {
      if (visited[i] == 0 && key[i] > max) {
         max = key[i];
         index = i;
      }
   }
   return index;
}

int maximumSpanningTree(int graph[100][100], int n) {
   int key[100], visited[100];
   int totalCost = 0;

   for (int i = 0; i < n; i++) {
      key[i] = INT_MIN;
      visited[i] = 0;
   }

   key[0] = 0;  // start from vertex 0

   for (int count = 0; count < n; count++) {
      int u = maxKey(key, visited, n);
      visited[u] = 1;
      totalCost += key[u];

      for (int v = 0; v < n; v++) {
         if (graph[u][v] != 0 && visited[v] == 0 && graph[u][v] > key[v]) {
            key[v] = graph[u][v];
         }
```

```
        }
    }

    return totalCost;
}

int main() {
    int n;
    int graph[100][100];

    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int result = maximumSpanningTree(graph, n);
    printf("Maximum Spanning Weight: %d", result);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-24.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
7
0 0 7 5 0 0 0
0 0 8 5 0 0 0
7 8 0 9 7 0 0
5 0 9 0 15 6 0
0 5 7 15 0 8 9
0 0 0 6 8 0 11
0 0 0 0 9 11 0
Maximum Spanning Weight: 59
```

WEEK 9 :

I. Given a graph, Design an algorithm and implement it using a program to implement Floyd-Warshall all pair shortest path algorithm

```
#include<iostream>
#include<vector>
using namespace std;

int main() {
   int v;
   cin >> v;

   vector<vector<int>> graph(v, vector<int>(v));
   int INF = 1000000000;

   // Input and conversion
   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         cin >> graph[i][j];

         if(i != j && graph[i][j] == 0) {
            graph[i][j] = INF;  // no edge
         }
      }
   }

   // Floyd-Warshall
   for(int k = 0; k < v; k++) {
      for(int i = 0; i < v; i++) {
         for(int j = 0; j < v; j++) {
            if(graph[i][k] + graph[k][j] < graph[i][j]) {
               graph[i][j] = graph[i][k] + graph[k][j];
            }
         }
      }
   }  // Output
   cout << "Shortest path matrix:\n";
   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         if(graph[i][j] == INF)
            cout << "INF ";
         else
            cout << graph[i][j] << " ";
      }
      cout << endl;
   }

   return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-25.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 3 0 0 20
3 0 5 0 0
0 5 0 2 0
0 0 2 0 4
20 0 0 4 0
Shortest path matrix:
0 3 8 10 14
3 0 5 7 11
8 5 0 2 6
10 7 2 0 4
14 11 6 4 0
```

/*I. Given a graph, Design an algorithm and implement it using a program to implement Floyd-Warshall all pair shortest path algorithm*/

```
#include<iostream>
#include<vector>
using namespace std;

int main() {
   int v;
   cin >> v;

   vector<vector<int>> graph(v, vector<int>(v));
   int INF = 1000000000;

   // Input and conversion
   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         cin >> graph[i][j];

         if(i != j && graph[i][j] == 0) {
            graph[i][j] = INF;  // no edge
         }
      }
   }

   // Floyd-Warshall
   for(int k = 0; k < v; k++) {
      for(int i = 0; i < v; i++) {
         for(int j = 0; j < v; j++) {
            if(graph[i][k] + graph[k][j] < graph[i][j]) {
               graph[i][j] = graph[i][k] + graph[k][j];
            }
         }
      }
   }

   // Output
   cout << "Shortest path matrix:\n";
   for(int i = 0; i < v; i++) {
      for(int j = 0; j < v; j++) {
         if(graph[i][j] == INF)
            cout << "INF ";
         else
            cout << graph[i][j] << " ";
      }
      cout << endl;
   }

   return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-25.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
0 3 0 0 20
3 0 5 0 0
0 5 0 2 0
0 0 2 0 4
20 0 0 4 0
Shortest path matrix:
0 3 8 10 14
3 0 5 7 11
8 5 0 2 6
10 7 2 0 4
14 11 6 4 0
```

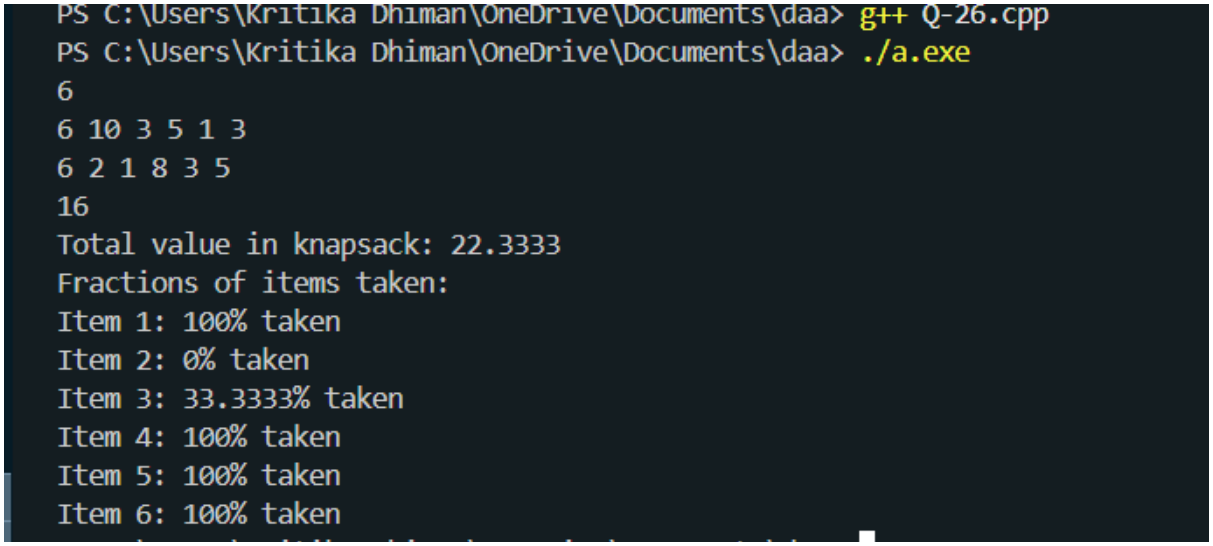
/*II. Given a knapsack of maximum capacity w. N items are provided, each having its own value and weight. You have to Design an algorithm and implement it using a program to find the list of the selected items such that the final selected content has weight w and has

maximum value. You can take fractions of items,i.e. the items can be broken into smaller pieces so that you have to carry only a fraction xi of item i, where 0 ≤xi≤ 1.*/

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, w;
    cin >> n;

    vector<double> weight(n), value(n);


    for (int i = 0; i < n; i++) {
        cin >> weight[i];
    }


    for (int i = 0; i < n; i++) {
        cin >> value[i];
    }

    cin >> w;

    vector<pair<double, int>> items;


    for (int i = 0; i < n; i++) {
        if (weight[i] == 0) continue;
        items.push_back({value[i] / weight[i], i});
    }


    sort(items.begin(), items.end(), greater<pair<double, int>>());

    double totalValue = 0.0;
    vector<double> fractions(n, 0.0);

    for (auto item : items) {
        int idx = item.second;

        if (weight[idx] <= w) {
            fractions[idx] = 1.0;
            totalValue += value[idx];
            w -= weight[idx];
        } else {
            fractions[idx] = (double)w / weight[idx];
            totalValue += value[idx] * fractions[idx];
            w = 0;
            break;
        }
```

```
    }

    cout << "Total value in knapsack: " << totalValue << endl;
    cout << "Fractions of items taken:\n";

    for (int i = 0; i < n; i++) {
        cout << "Item " << i + 1 << ": " << fractions[i] * 100 << "% taken\n";
    }

    return 0;
}
```

OUTPUT:



/*III. Given an array of elements. Assume arr[i] represents the size of file i. Write an algorithm and a
program to merge all these files into single file with minimum computation. For given two files

A and B with sizes m and n, computation cost of merging them is O(m+n). (Hint: use greedy approach)*/

```
#include <bits/stdc++.h>
using namespace std;

int main() {
   int n;
   cout << "Enter number of files: ";
   cin >> n;

   vector<int> fileSizes(n);
   cout << "Enter sizes of files:\n";
   for (int i = 0; i < n; i++) {
      cin >> fileSizes[i];
   }

   priority_queue<int, vector<int>, greater<int>> pq(fileSizes.begin(), fileSizes.end());

   int totalCost = 0;

   while (pq.size() !=1) {
      int first = pq.top();
      pq.pop();
      int second = pq.top();
      pq.pop();

      int mergeCost = first + second;
      totalCost += mergeCost;

      pq.push(mergeCost);
   }

   cout << "Minimum computation: " << totalCost << endl;

   return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-27.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
Enter number of files: 10
Enter sizes of files:
10 5 100 50 20 15 5 20 100 10
Minimum computation: 895
```

WEEK 10:

/*I. Given a list of activities with their starting time and finishing time. Your goal is to select maximum number of activities that can be performed by a single person such that selected activities must be non-conflicting. Any activity is said to be non-conflicting if starting time of an
activity is greater than or equal to the finishing time of the other activity. Assume that a person
can only work on a single activity at a time.
Input Format:
First line of input will take number of activities N.
Second line will take N space-separated values defining starting time for all the N activities.
Third line of input will take N space-separated values defining finishing time for all the N activities.
Output Format:
Output will be the number of non-conflicting activities and the list of selected activities.*/

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
   int n;
   cin>>n;
   vector<int>time_req(n);
   vector<int>finish(n);

   for(int i=0;i<n;i++)
   {
      cin>>time_req[i]>>finish[i];
   }
   vector<vector<int>> task;
   for(int i=0;i<n;i++)
   {
      task.push_back({finish[i],time_req[i],i+1});
   }
   sort(task.begin(),task.end());
   int current_time=0;
   int count=0;
   vector<int>ans;
   priority_queue<pair<int,int>> pq;
   for(auto it: task)
   {
      int finish_time=it[0];
      int time_r=it[1];
      int pos=it[2];

         pq.push({time_r,pos});
         current_time+=time_r;
      if(current_time> finish_time)
      {
         auto t=pq.top();
         pq.pop();
```

```
            current_time-=t.first;
        }
    }
    cout<<" maximum task performed: "<<pq.size()<<endl;
    while(!pq.empty())
    {
        cout<<pq.top().second<<" ";
        pq.pop();
    }
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-28.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
10
1 3 0 5 3 5 8 8 2 12
4 5 6 7 9 9 11 12 14 16
 maximum task performed: 4
3 5 1 2
```

/*II. Given a long list of tasks. Each task takes specific time to accomplish it and each task has a

deadline associated with it. You have to design an algorithm and implement it using a program to
find maximum number of tasks that can be completed without crossing their deadlines and also
find list of selected tasks.*/

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;

int main() {
   int n;
   cin >> n;

   vector<int> time(n), deadline(n);

   for(int i = 0; i < n; i++)
      cin >> time[i];

   for(int i = 0; i < n; i++)
      cin >> deadline[i];

   // store (deadline, time, index)
   vector<vector<int>> tasks;

   for(int i = 0; i < n; i++) {
      tasks.push_back({deadline[i], time[i], i + 1});
   }

   // sort by deadline
   sort(tasks.begin(), tasks.end());

   vector<int> selected;
   int totalTime = 0;

   for(int i = 0; i < n; i++) {
      totalTime += tasks[i][1];
      selected.push_back(i);

      // find max time task if needed
      int maxTime = -1, maxIndex = -1;

      for(int j = 0; j < selected.size(); j++) {
         if(tasks[selected[j]][1] > maxTime) {
            maxTime = tasks[selected[j]][1];
            maxIndex = j;
         }
      }
```

```
        // if deadline exceeded → remove longest task
        if(totalTime > tasks[i][0]) {
            totalTime -= tasks[selected[maxIndex]][1];
            selected.erase(selected.begin() + maxIndex);
        }
    }

    cout << "Max number of tasks = " << selected.size() << endl;

    cout << "Selected task numbers : ";
    for(int i = 0; i < selected.size(); i++) {
        cout << tasks[selected[i]][2];
        if(i != selected.size() - 1)
            cout << ", ";
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-29.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
7
2 1 3 2 2 2 1
2 3 8 6 2 5 3
Max number of tasks = 4
Selected task numbers : 2, 7, 6, 4
```

/*III.Given an unsorted array of elements, design an algorithm and implement it using a program to

find whether majority element exists or not. Also find median of the array. A majority element is
an element that appears more than n/2 times, where n is the size of array.
Input Format:
First line of input will give size n of array.
Second line of input will take n space-separated elements of array.
Output Format:
First line of output will be 'yes' if majority element exists, otherwise print 'no'.
Second line of output will print median of the array.*/

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cin>>n;
    vector<int> arr(n);
    for(int i=0;i<n;i++){
        cin>>arr[i];
    }
    unordered_map<int,int>feq;
    for(int i=0;i<n;i++){
        feq[arr[i]]++;
    }
    int majority = -1;
    for(auto it:feq){
        if(it.second > n/2){
            majority = it.first;
            break;
        }

    }
    if(majority != -1){
        cout<<"yes"<<endl;
    }
    else{
        cout<<"no"<<endl;
    }
    sort(arr.begin(),arr.end());
    double median;
    if(n%2==0){
        median = (arr[n/2-1]+arr[n/2])/2.0;
    }
    else{
        median = arr[n/2];
    }
    cout<<median<<endl;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-30.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
9
4 4 2 3 2 2 3 2 2
yes
2
```

WEEK 11:

/*I. Given a sequence of matrices, write an algorithm to find most efficient way to multiply these
matrices together. To find the optimal solution, you need to find the order in which these matrices should be multiplied.
Input Format:
First line of input will take number of matrices n that you need to multiply.
For each line i in n, take two inputs which will represent dimensions aXb of matrix i.
Output Format:
Output will be the minimum number of operations that are required to multiply the list of matrices.*/

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;

    vector<pair<int, int>> dimensions(n);
    for (int i = 0; i < n; i++) {
        cin >> dimensions[i].first >> dimensions[i].second;
    }

    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int length = 2; length <= n; length++) {
        for (int i = 0; i <= n - length; i++) {
            int j = i + length - 1;
            dp[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j] + dimensions[i].first * dimensions[k].second *
dimensions[j].second;
                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }

    cout << "Minimum number of operations: " << dp[0][n - 1] << endl;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-31.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
3
10 30
30 5
5 60
Minimum number of operations: 4500
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>
```

/*II. Given a set of available types of coins. Let suppose you have infinite supply of each type of coin.

For a given value N, you have to Design an algorithm and implement it using a program to find
number of ways in which these coins can be added to make sum value equals to N.
Input Format:
First line of input will take number of coins that are available.
Second line of input will take the value of each coin.
Third line of input will take the value N for which you need to find sum.
Output Format:
Output will be the number of ways.*/

```
#include <bits/stdc++.h>
using namespace std;
int countWaysToMakeChange(vector<int>& coins, int N) {
    vector<int> dp(N + 1, 0);
    dp[0] = 1; // Base case: There is one way to make change for 0, which is to use no coins.

    for (int coin : coins) {
        for (int j = coin; j <= N; j++) {
            dp[j] += dp[j - coin];
        }
    }

    return dp[N];
}
int main() {
    int numCoins;
    cin >> numCoins;

    vector<int> coins(numCoins);
    for (int i = 0; i < numCoins; i++) {
        cin >> coins[i];
    }

    int N;
    cin >> N;

    int result = countWaysToMakeChange(coins, N);
    cout << "Number of ways to make change: " << result << endl;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-32.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
4
2 5 6 3
10
Number of ways to make change: 5
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>
```

/*III.Given a set of elements, you have to partition the set into two subsets such that the sum of

elements in both subsets is same. Design an algorithm and implement it using a program to solve
this problem.
Input Format:
First line of input will take number of elements n present in the set.
Second line of input will take n space-separated elements of the set.
Output Format:
Output will be 'yes' if two such subsets found otherwise print 'no'.*/

```
#include <bits/stdc++.h>
using namespace std;
bool canPartition(vector<int>& nums) {
    int totalSum = accumulate(nums.begin(), nums.end(), 0);


    if (totalSum % 2 != 0) {
        return false;
    }

    int target = totalSum / 2;
    vector<bool> dp(target + 1, false);
    dp[0] = true;

    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }

    return dp[target];
}
int main() {
    int n;
    cin >> n;

    vector<int> nums(n);
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    if (canPartition(nums)) {
        cout << "yes" << endl;
    } else {
        cout << "no" << endl;
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-33.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
7
1 5 4 11 5 14 10
yes
```

WEEK 12:

/*I. Given two sequences, Design an algorithm and implement it using a program to find the length
of longest subsequence present in both of them. A subsequence is a sequence that appears in the
same relative order, but not necessarily contiguous.
Input Format:
First input line will take character sequence 1.
Second input line will take character sequence 2.
Output Format:
Output will be the longest common subsequence along with its length.*/

```
#include <bits/stdc++.h>
using namespace std;

pair<int, string> longestCommonSubsequence(const string& s1, const string& s2) {
    int n = s1.length();
    int m = s2.length();

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    int lcsLength = dp[n][m];
    string lcs = "";
    int i = n, j = m;

    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1]) {
            lcs = s1[i - 1] + lcs;
            i--;
            j--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }

    return {lcsLength, lcs};
}

int main() {
```

```
    string s1, s2;
    getline(cin, s1);
    getline(cin, s2);

    pair<int, string> result = longestCommonSubsequence(s1, s2);
    cout << "Length of Longest Common Subsequence: " << result.first << endl;
    cout << "Longest Common Subsequence: " << result.second << endl;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-34.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
AGGTAB
GXTXAYB
Length of Longest Common Subsequence: 4
Longest Common Subsequence: GTAB
```

/*II. Given a knapsack of maximum capacity w. N items are provided, each having its own value and

weight. Design an algorithm and implement it using a program to find the list of the selected items such that the final selected content has weight <= w and has maximum value. Here, you cannot break an item i.e. either pick the complete item or don't pick it. (0-1 property).
Input Format:
First line of input will provide number of items n.
Second line of input will take n space-separated integers describing weights for all items.
Third line of input will take n space-separated integers describing value for each item.
Last line of input will give the knapsack capacity.
Output Format:
Output will be maximum value that can be achieved and list of items selected along with their weight and value.*/

```
#include <bits/stdc++.h>
using namespace std;
pair<int, vector<int>> knapsack(int W, vector<int>& weights, vector<int>& values) {
    int n = weights.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (weights[i - 1] <= w) {
                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weights[i - 1]] + values[i - 1]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    int maxValue = dp[n][W];
    vector<int> selectedItems;
    int w = W;

    for (int i = n; i > 0 && maxValue > 0; i--) {
        if (maxValue != dp[i - 1][w]) {
            selectedItems.push_back(i - 1);
            maxValue -= values[i - 1];
            w -= weights[i - 1];
        }
    }

    reverse(selectedItems.begin(), selectedItems.end());
    return {dp[n][W], selectedItems};
}
int main() {
    int n;
    cin >> n;

    vector<int> weights(n), values(n);
    for (int i = 0; i < n; i++) {
        cin >> weights[i];
```

```
    }
    for (int i = 0; i < n; i++) {
        cin >> values[i];
    }

    int W;
    cin >> W;

    pair<int, vector<int>> result = knapsack(W, weights, values);
    cout << "Maximum value in Knapsack: " << result.first << endl;
    cout << "Selected items (index, weight, value):" << endl;
    for (int index : result.second) {
        cout << "Item " << index << ": Weight = " << weights[index] << ", Value = " <<
values[index] << endl;
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-35.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
5
2 3 3 4 6
1 2 5 9 4
10
Maximum value in Knapsack: 16
Selected items (index, weight, value):
Item 1: Weight = 3, Value = 2
Item 2: Weight = 3, Value = 5
Item 3: Weight = 4, Value = 9
```

/*III.Given a string of characters, design an algorithm and implement it using a program to print all

possible permutations of the string in lexicographic order.
Input Format:
String of characters is provided as an input.
Output Format:
Output will be the list of all possible permutations in lexicographic order*/

```
#include <bits/stdc++.h>
using namespace std;
void permute(string str, int l, int r, vector<string>& result) {
    if (l == r) {
        result.push_back(str);
    } else {
        for (int i = l; i <= r; i++) {
            swap(str[l], str[i]);
            permute(str, l + 1, r, result);
            swap(str[l], str[i]);
        }
    }
}
int main() {
    string str;
    cin >> str;

    vector<string> result;
    permute(str, 0, str.length() - 1, result);

    sort(result.begin(), result.end());
    cout << "Permutations in lexicographic order:" << endl;
    for (const string& perm : result) {
        cout << perm << endl;
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-36.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
CAB
Permutations in lexicographic order:
ABC
ACB
BAC
BCA
CAB
CBA
```

WEEK 13:

/*I. Given an array of characters, you have to find distinct characters from this array. Design an
algorithm and implement it using a program to solve this problem using hashing. (Time Complexity = O(n))*/

```
#include<bits/stdc++.h>

using namespace std;
int main() {
   int n;
   cout << "Enter the size of the character array: ";
   cin >> n;

   vector<char> charArray(n);
   cout << n;
   for (int i = 0; i < n; i++) {
      cin >> charArray[i];
   }

   unordered_map<char, int> frequencyMap;
   for (char c : charArray) {
      frequencyMap[c]++;
   }

   vector<pair<char, int>> distinctChars(frequencyMap.begin(), frequencyMap.end());
   sort(distinctChars.begin(), distinctChars.end());

   cout << "Distinct characters and their frequencies in alphabetical order:" << endl;
   for (const auto& pair : distinctChars) {
      cout << pair.first << ": " << pair.second << endl;
   }

   return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-37.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
Enter the size of the character array: 20
20
a e d e f j t t z a z f t a e e k a e q
Distinct characters and their frequencies in alphabetical order:
a: 4
d: 1
e: 5
f: 2
j: 1
k: 1
q: 1
t: 3
z: 2
```

/*II. Given an array of integers of size n, design an algorithm and write a program to check whether

this array contains duplicate within a small window of size k < n.*/

```
#include <iostream>
#include <unordered_set>
#include <vector>
using namespace std;

int main() {
   int T;
   cin >> T;

   while (T--) {
      int n, k;
      cin >> n;

      vector<int> arr(n);
      for (int i = 0; i < n; i++) {
         cin >> arr[i];
      }

      cin >> k;

      unordered_set<int> windowSet;
      bool duplicateFound = false;

      for (int i = 0; i < n; i++) {
         // Check duplicate in current window
         if (windowSet.find(arr[i]) != windowSet.end()) {
            duplicateFound = true;
            break;
         }

         windowSet.insert(arr[i]);

         // Maintain window size of k
         if (i >= k) {
            windowSet.erase(arr[i - k]);
         }
      }

      if (duplicateFound)
         cout << "Duplicate present in window "<<k << endl;
      else
         cout << "Duplicate not present in window " <<k<< endl;
   }

   return 0;
   }
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-38.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
2
10
1 2 3 4 1 2 3 4 1 2
3
Duplicate not present in window 3
12
1 2 3 1 2 3 1 2 3 1 2 3
4
Duplicate present in window 4
```

/*III.Given an array of nonnegative integers, Design an algorithm and implement it using a program

to find two pairs (a,b) and (c,d) such that a*b = c*d, where a, b, c and d are distinct elements of
array.
Input Format:
First line of input will give size of array n.
Second line of input will give n space-separated array elements.
Output Format:
First line of output will give pair (a,b)
Second line of output will give pair (c,d).*/

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    unordered_map<int, pair<int, int>> productMap;
    bool found = false;

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int product = arr[i] * arr[j];

            if (productMap.find(product) != productMap.end()) {
                auto& p = productMap[product];
                if (p.first != arr[i] && p.second != arr[i] && p.first != arr[j] && p.second != arr[j])
{
                    cout << "Pair 1: (" << p.first << ", " << p.second << ")" << endl;
                    cout << "Pair 2: (" << arr[i] << ", " << arr[j] << ")" << endl;
                    found = true;
                    break;
                }
            } else {
                productMap[product] = {arr[i], arr[j]};
            }
        }
        if (found) break;
    }if (!found) {
        cout << "No such pairs found." << endl;
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-39.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
10
31 23 4 1 39 2 20 27 8 10
Pair 1: (4, 2)
Pair 2: (1, 8)
```

WEEK 14:

/*I. Given a number n, write an algorithm and a program to find nth ugly number. Ugly numbers are
those numbers whose only prime factors are 2, 3 or 5. The sequence 1, 2, 3, 4, 5, 6, 8, 9, 10, 12,
15, 16, 18, 20, 24,..... is sequence of ugly numbers.
Input:
First line of input will give number of test cases T.
For each test case T, enter a number n.
Output:
There will be T output lines.
For each test case T, Output will be nth ugly number*/

```
#include <bits/stdc++.h>
using namespace std;
int nthUglyNumber(int n) {
   vector<int> uglyNumbers(n);
   uglyNumbers[0] = 1;

   int i2 = 0, i3 = 0, i5 = 0;
   for (int i = 1; i < n; i++) {
      int nextUgly = min({uglyNumbers[i2] * 2, uglyNumbers[i3] * 3, uglyNumbers[i5] *
5});
      uglyNumbers[i] = nextUgly;

      if (nextUgly == uglyNumbers[i2] * 2) i2++;
      if (nextUgly == uglyNumbers[i3] * 3) i3++;
      if (nextUgly == uglyNumbers[i5] * 5) i5++;
   }

   return uglyNumbers[n - 1];
}
int main() {
   int T;
   cin >> T;

   while (T--) {
      int n;
      cin >> n;
      cout << nthUglyNumber(n) << endl;
   }

   return 0;
}
```

OUTPUT:

```
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-40.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
3
11
15
8
9
15
24
```

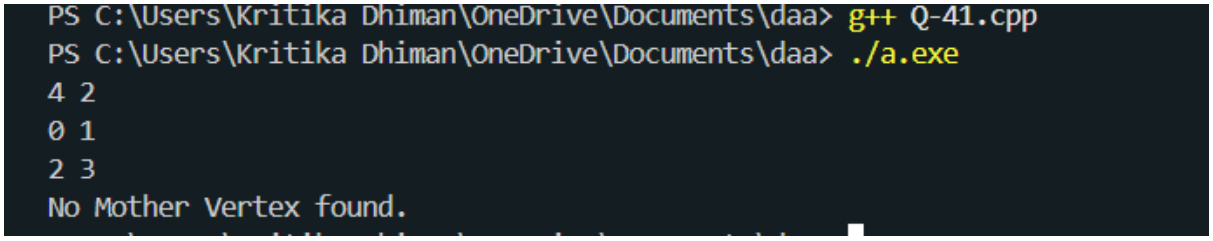
/*II.Given a directed graph, write an algorithm and a program to find mother vertex in a graph. A

mother vertex is a vertex v such that there exists a path from v to all other vertices of the graph.
Input:
Graph in the form of adjacency matrix or adjacency list is provided as an input.
Output:
Output will be the mother vertex number.*/

```
#include <bits/stdc++.h>
using namespace std;
void DFS(int v, vector<vector<int>>& adjList, vector<bool>& visited) {
   visited[v] = true;
   for (int neighbor : adjList[v]) {
      if (!visited[neighbor]) {
         DFS(neighbor, adjList, visited);
      }
   }
}
int findMotherVertex(vector<vector<int>>& adjList) {
   int V = adjList.size();
   vector<bool> visited(V, false);
   int lastVisited = 0;

   for (int i = 0; i < V; i++) {
      if (!visited[i]) {
         DFS(i, adjList, visited);
         lastVisited = i;
      }
   }

   fill(visited.begin(), visited.end(), false);
   DFS(lastVisited, adjList, visited);

   for (bool v : visited) {
      if (!v) return -1; // No mother vertex found
   }
   return lastVisited;
}
int main() {
   int V, E;
   cin >> V >> E;

   vector<vector<int>> adjList(V);
   for (int i = 0; i < E; i++) {
      int u, v;
      cin >> u >> v;
      adjList[u].push_back(v);
   }

   int motherVertex = findMotherVertex(adjList);
   if (motherVertex != -1) {
```

```
        cout << "Mother Vertex: " << motherVertex << endl;
    } else {
        cout << "No Mother Vertex found." << endl;
    }

    return 0;
}
```

OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-41.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
4 2
0 1
2 3
No Mother Vertex found.

Week 1:

1.Given an array of nonnegative integers, design a linear algorithm and implement it using a program to find whether given key element is present in the array or not. Also, find total number of comparisons for each input case. (Time Complexity = O(n), where n is the size of input)

Source code:

```
#include <iostream>
#include <vector>
using namespace std;
void linear_search(const vector<int>& v, int key) {
   int comp = 0;
   bool found = false;
   for (int i = 0; i < v.size(); i++) {
      comp++;
      if (v[i] == key) {
         cout << "Present " << comp << endl;
         found = true;
         break;
      }
   }
   if (!found) {
      cout << "Not Present " << comp << endl;
   }
}
int main() {
   int t;
   cin >> t;
   while (t--) {
      int n;
      cin >> n;
      vector<int> v(n);
      for (int i = 0; i < n; i++) {
         cin >> v[i];
      }
      int key;
      cin >> key;
      linear_search(v, key);
   }return 0;
}
```

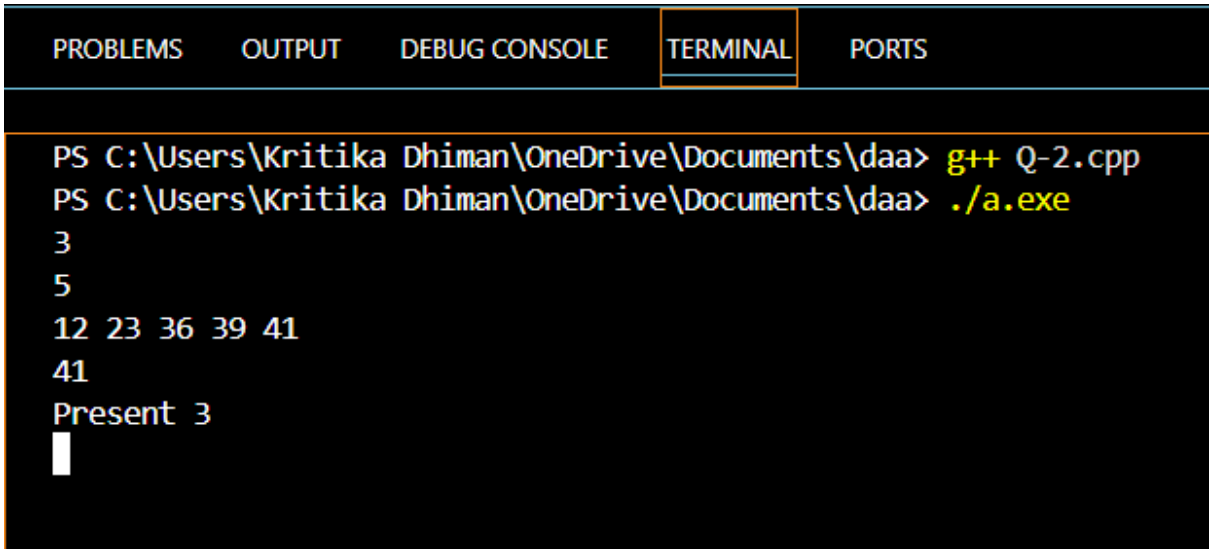
Sample Output:

Week 1:

II. Given an already sorted array of positive integers, design an algorithm and implement it using a program to find whether given key element is present in the array or not. Also, find total number of comparisons for each input case. (Time Complexity = O(nlogn), where n is the size of input).

```
#include <iostream>
#include <vector>
using namespace std;
void binary_search(const vector<int>& v, int key) {
   int start = 0, end = v.size() - 1;
   int comp = 0;
   bool found = false;
   while (start <= end) {
      int mid = (start + end) / 2;
      comp++;
      if (v[mid] == key) {
         cout << "Present " << comp << endl;
         found = true;
         break;
      } else if (key > v[mid]) {
         start = mid + 1;
      }
      else {
         end = mid - 1;
      }
   } if (!found) {
      cout << "Not Present " << comp << endl;
   }
}int main() {
   int t;
   cin >> t;
   while (t--) {
      int n;
      cin >> n;
      vector<int> v(n);
      for (int i = 0; i < n; i++) {
         cin >> v[i];
      }int key;
      cin >> key;
       binary_search(v, key);
   }return 0;
}
```

Sample Output:



WEEK1:

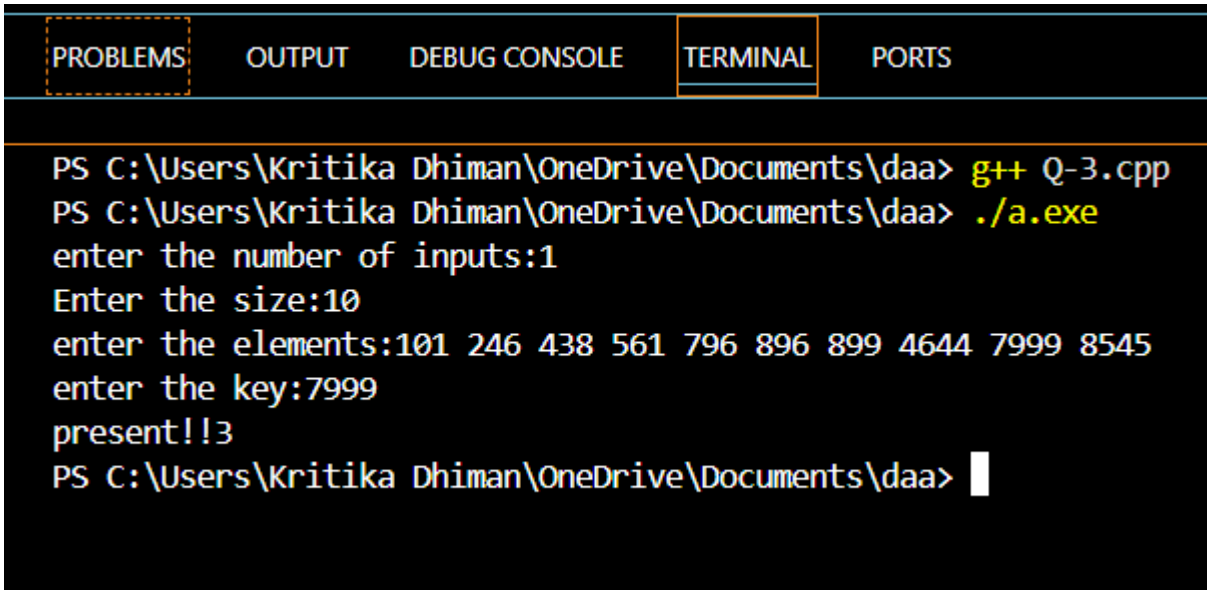
III. Given an already sorted array of positive integers, design an algorithm and implement it using a program to find whether a given key element is present in the sorted array or not. For an array arr[n], search at the indexes arr[0], arr[2], arr[4],.....,arr[2k] and so on. Once the interval (arr[2k] < key < arr[2k+1]) is found, perform a linear search operation from the index 2k to find the element key. (Complexity < O(n), where n is the number of elements need to be scanned for searching):

Jump Search

```
#include<iostream>
using namespace std;
#include<vector>
#include<math.h>
void linear_search(vector <int> &arr,int st,int end ,int size,int key){
   int c=0;
   int i=0;
   for(i=st ; i<min(end,size);i++){
      c++;
      if(arr[i]==key){
         cout<<"present!!"<<c;
         break;
      }
   }
}
void jump_search(vector <int>&vec,int key, int size){
   int i=0;
   int st=0,wid=sqrt(size),end=st+wid;
   while(end<size && vec[end-1]<key){
      st = end;
      end+=wid;
   }
   linear_search(vec,st,end,size,key);
}
int main(){
   int t;
   cout<<"enter the number of inputs:";
   cin>>t;
   int i=0,key;
   while(i<t){
      int n,x;
      cout<<"Enter the size:";
      cin>>n;
```

```
        vector<int>vec;
        int j=0;
        cout<<"enter the elements:";
        while(j<n){
            cin>>x;
            vec.push_back(x);
            j++;
        }
        cout<<"enter the key:";
        cin>>key;
        jump_search(vec,key,n);
        i++;
    }
}
```

Sample Output:



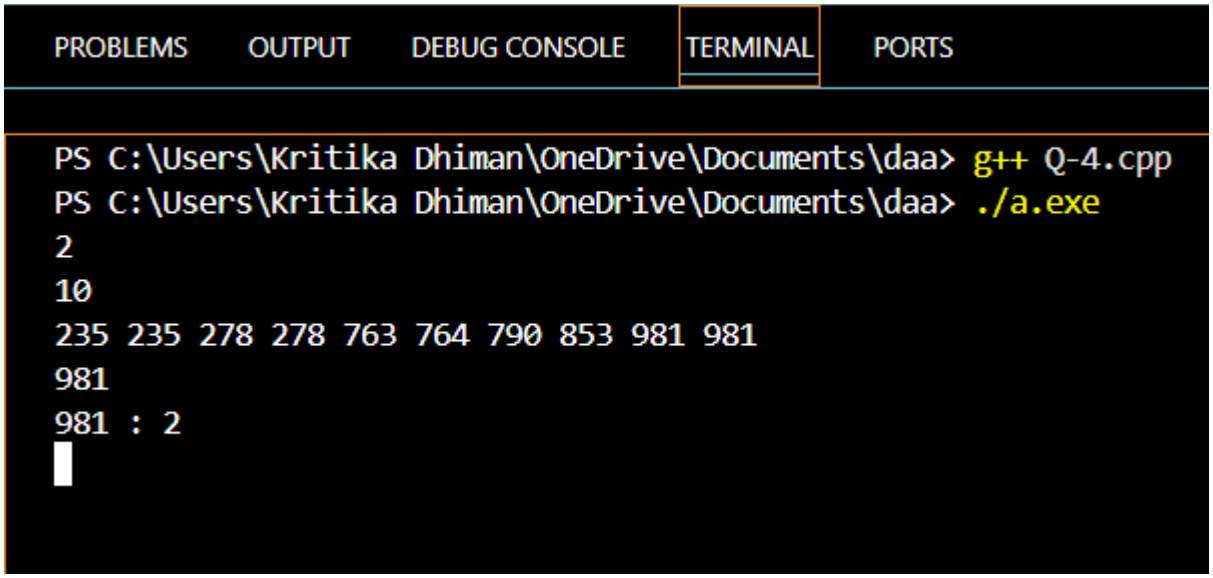
WEEK2:

I. Given a sorted array of positive integers containing few duplicate elements, design an algorithm and implement it using a program to find whether the given key element is present in the array or not. If present, then also find the number of copies of given key. (Time Complexity = O(log n))

```
#include<iostream>
using namespace std;
int first_occurence(int arr[],int n,int key){
   int s=0,e=n-1,mid=-1,ans=-1;

   while(s<=e){
      mid=(s+e)/2;
      if(arr[mid]==key){
         ans=mid;
         e=mid-1;
      }
      else if(key>arr[mid]){
         s=mid+1;
      }
      else{
         e=mid-1;
      }
   }
   return ans;
}
int last_occurence(int arr[],int n,int key){
   int s=0,e=n-1,mid=-1,ans=-1;

   while(s<=e){
      mid=(s+e)/2;
      if(arr[mid]==key){
         ans=mid;
         s=mid+1;
      }
      else if(key>arr[mid]){
         s=mid+1;
      }
      else{
         e=mid-1;
      }
   }
   return ans;
```

```
}

int main(){
   int t;
   cin>>t;
   while(t--){
      int n;
      cin>>n;
      int arr[n];
      for(int i=0;i<n;i++){
         cin>>arr[i];
      }
      int key;
      cin>>key;
      int fi=first_occurence(arr,n,key);
      int li=last_occurence(arr,n,key);
    if(li==-1||fi==-1){
      cout<<"the key is not present";
    }
    else{
      cout<<key<<" : "<<(li-fi)+1<<endl;
    }

   }
}
```

Sample Output:

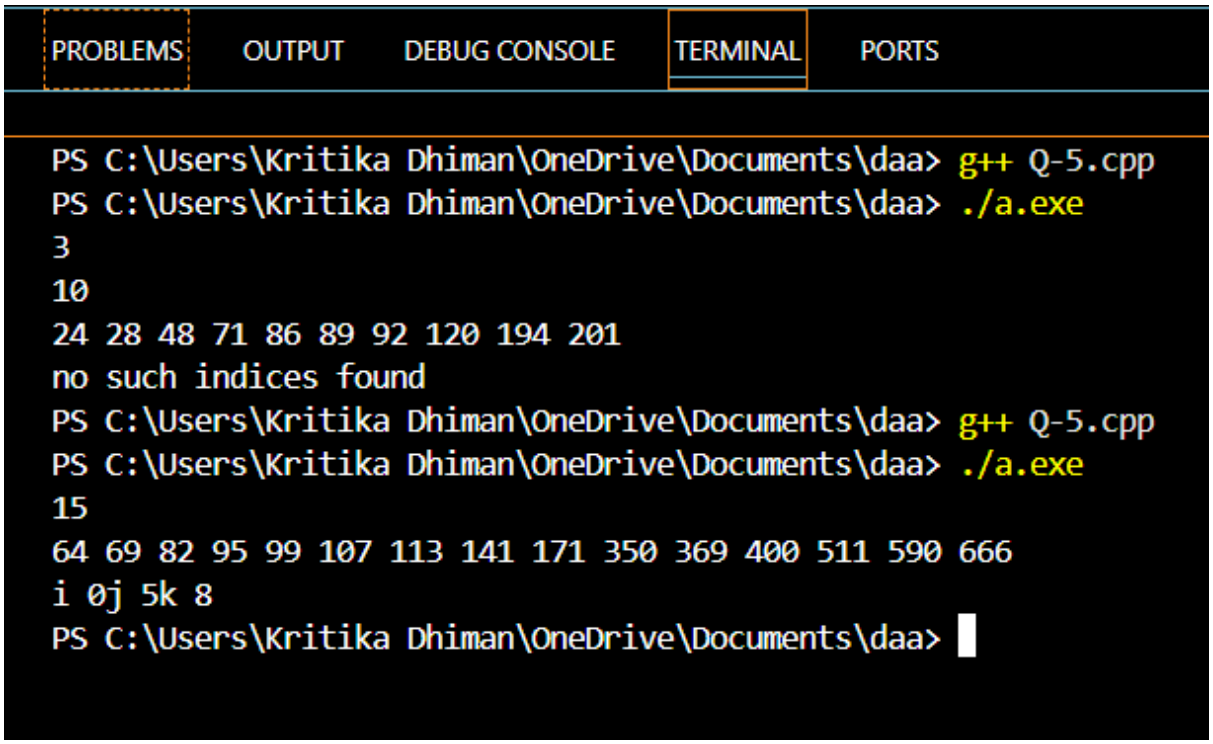


WEEK2:

II. Given a sorted array of positive integers, design an algorithm and implement it using a program to find three indices i, j, k such that arr[i] + arr[j] = arr[k].

```
#include<iostream>
using namespace std;
int main(){
   int n;
   cin>>n;

   int arr[n];
   for(int i=0;i<n;i++){
      cin>>arr[i];
   }

   bool found = false;

   for(int k=n-1;k>=0;k--){
      int i=0,j=k-1;
      while(i<j){
         if(arr[i]+arr[j]==arr[k]){
            cout<<"i "<<i<<"j "<<j<<"k "<<k<<endl;
            found = true;
            break;
         }else if ( arr[i]+arr[j]<arr[k]){
            i++;
         }
         else{
            j--;
         }

      }
   }
 if(!found)
   cout<<"no such indices found";
 return 0;
}
```

OUTPUT:

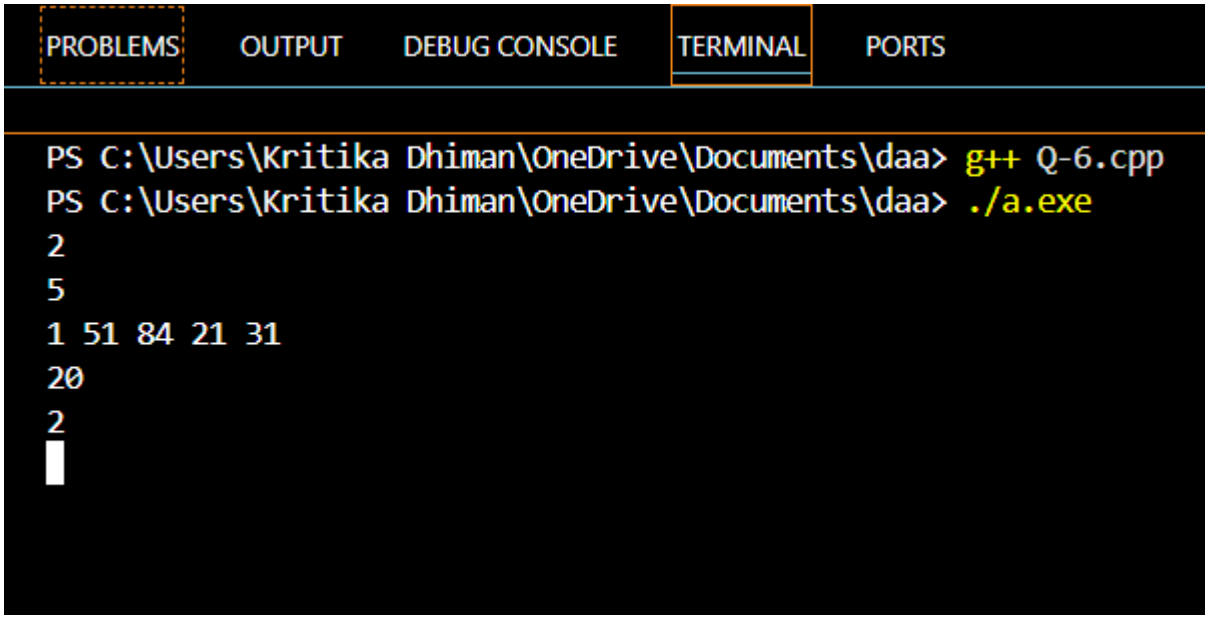

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-5.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
3
10
24 28 48 71 86 89 92 120 194 201
no such indices found
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-5.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
15
64 69 82 95 99 107 113 141 171 350 369 400 511 590 666
i 0j 5k 8
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

WEEK2:

III.Given an array of nonnegative integers, design an algorithm and a program to count the number of pairs of integers such that their difference is equal to a given key, K.

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

int main() {
   int T;
   cin >> T;

   while (T--) {
      int n;
      cin >> n;

      vector<int> arr(n);
      for (int i = 0; i < n; i++) {
         cin >> arr[i];
      }

      int K;
      cin >> K;

      unordered_set<int> s;
      int count = 0;

      for (int i = 0; i < n; i++) {
         if (s.find(arr[i] + K) != s.end())
            count++;

         if (s.find(arr[i] - K) != s.end())
            count++;

         s.insert(arr[i]);
      } cout << count << endl;
   }

   return 0;
}
```

OUTPUT:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-6.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
2
5
1 51 84 21 31
20
2

WEEK3:

I. Given an unsorted array of integers, design an algorithm and a program to sort the array using insertion sort. Your program should be able to find number of comparisons and shifts (shifts - total number of times the array elements are shifted from their place) required for sorting the array.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
   int a[100];   // simple array
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }int comparisons = 0;
    int shifts = 0;
for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0) {
            comparisons++;          // comparison
            if (a[j] > key) {
                a[j + 1] = a[j];    // shift
                shifts++;
                j--;
            } else {
                break;
            }
        }
        a[j + 1] = key;
    }

    cout << "Sorted array:\n";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }cout << "\nComparisons: " << comparisons;
    cout << "\nShifts: " << shifts;

    return 0;
}
```

OUTPUT:

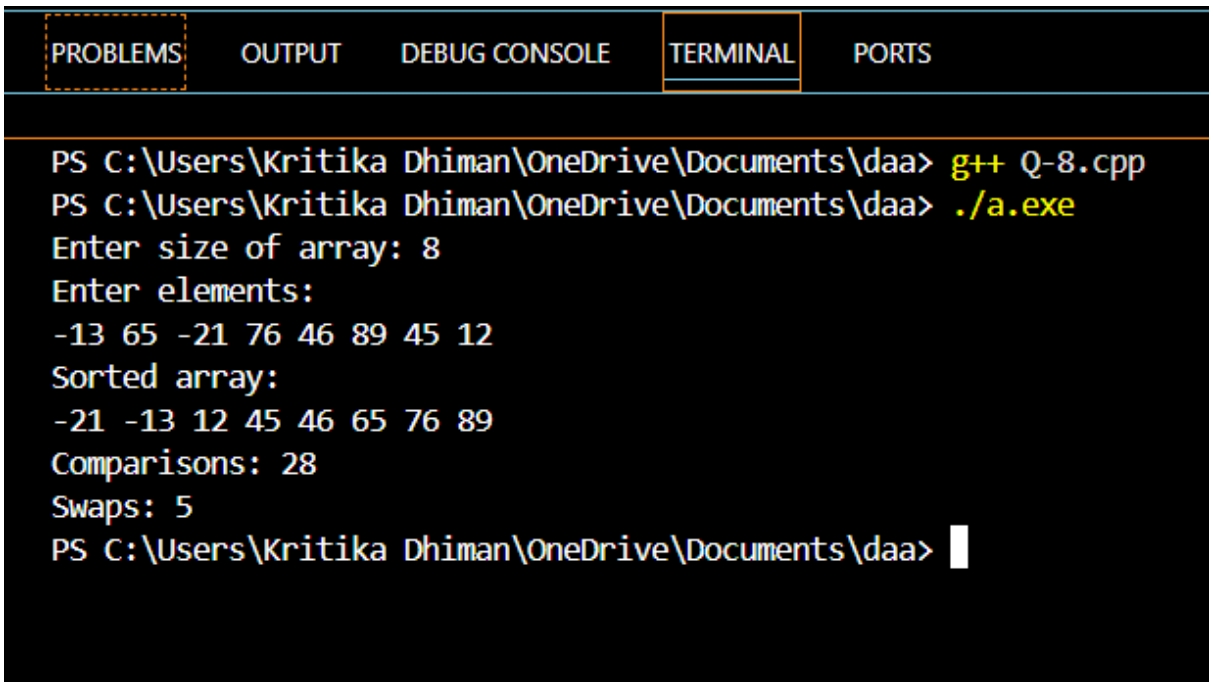

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-7.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
Enter size of array: 8
Enter elements:
-23 65 -31 76 46 89 45 32
Sorted array:
-31 -23 32 45 46 65 76 89
Comparisons: 19
Shifts: 13
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

WEEK3:

II. Given an unsorted array of integers, design an algorithm and implement a program to sort this array using selection sort. Your program should also find number of comparisons and number of swaps required.

```
#include <iostream>
using namespace std;
int main() {
   int n;
   cout << "Enter size of array: ";
   cin >> n;

   int a[100];
   cout << "Enter elements:\n";
   for (int i = 0; i < n; i++) {
      cin >> a[i];
   } int comparisons = 0;
   int swaps = 0;

   for (int i = 0; i < n - 1; i++) {
      int minIndex = i;

      for (int j = i + 1; j < n; j++) {
         comparisons++;              // comparison
         if (a[j] < a[minIndex]) {
            minIndex = j;
         }
      } if (minIndex != i) {            // swap only if needed
         int temp = a[i];
         a[i] = a[minIndex];
         a[minIndex] = temp;
         swaps++;
      }
   } cout << "Sorted array:\n";
   for (int i = 0; i < n; i++) {
      cout << a[i] << " ";
   }

   cout << "\nComparisons: " << comparisons;
   cout << "\nSwaps: " << swaps;

   return 0;
}
```

OUTPUT:

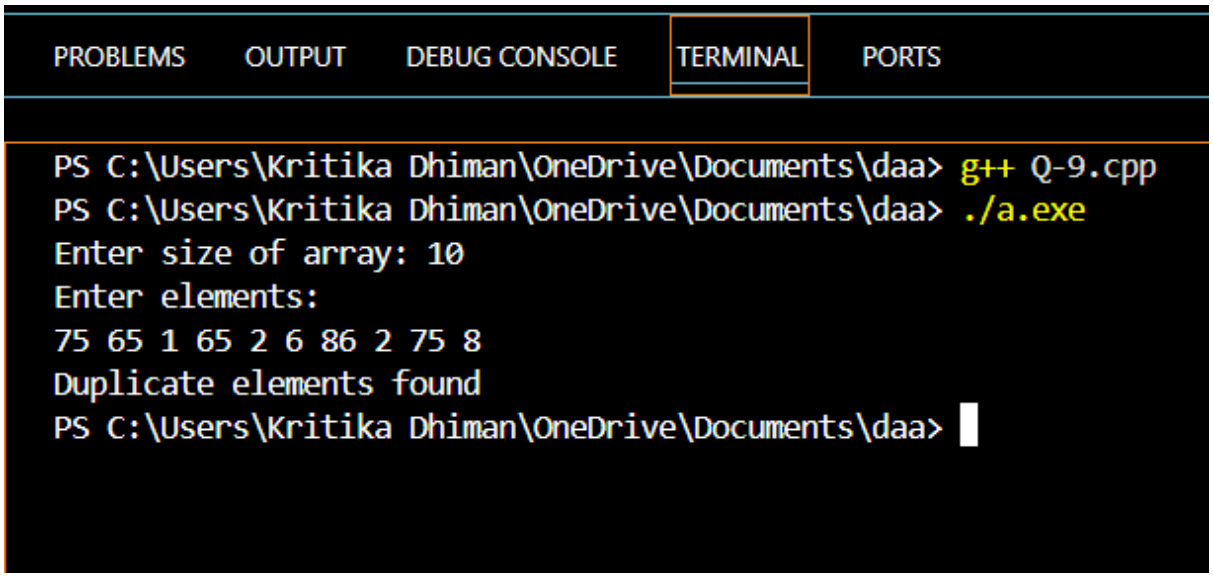

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-8.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
Enter size of array: 8
Enter elements:
-13 65 -21 76 46 89 45 12
Sorted array:
-21 -13 12 45 46 65 76 89
Comparisons: 28
Swaps: 5
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

WEEK3:

III.Given an unsorted array of positive integers, design an algorithm and implement it using a program to find whether there are any duplicate elements in the array or not. (use sorting) (Time Complexity = O(n log n))

```
#include <iostream>
using namespace std;
int main() {
   int n;
   cout << "Enter size of array: ";
   cin >> n;

   int a[100];
   cout << "Enter elements:\n";
   for (int i = 0; i < n; i++) {
      cin >> a[i];
   }
   for (int i = 0; i < n - 1; i++) {
      int minIndex = i;
      for (int j = i + 1; j < n; j++) {
         if (a[j] < a[minIndex]) {
            minIndex = j;
         }
      }
      if (minIndex != i) {
         int temp = a[i];
         a[i] = a[minIndex];
         a[minIndex] = temp;
      }
   }int duplicate = 0;
 for (int i = 0; i < n - 1; i++) {
      if (a[i] == a[i + 1]) {
         duplicate = 1;
         break;
      }
   }if (duplicate)
      cout << "Duplicate elements found";
   else
      cout << "No duplicate elements";

   return 0;
}
```

OUTPUT:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-9.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
Enter size of array: 10
Enter elements:
75 65 1 65 2 6 86 2 75 8
Duplicate elements found
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

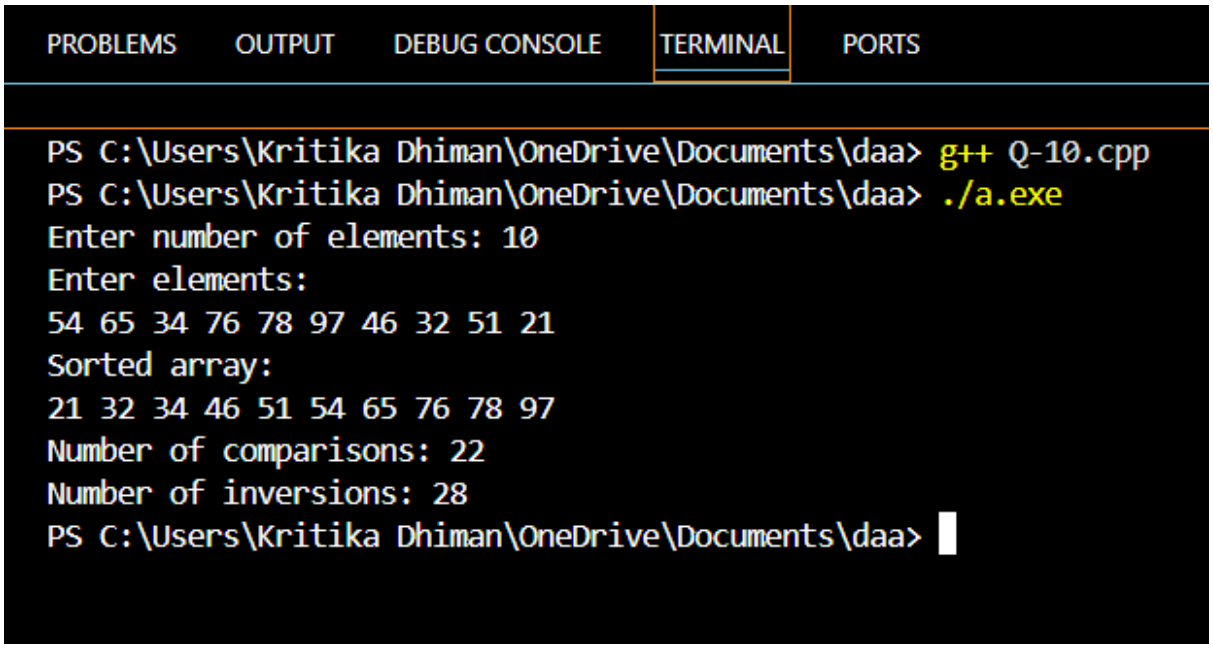
WEEK4:

I. Given an unsorted array of integers, design an algorithm and implement it using a program to sort an array of elements by dividing the array into two subarrays and combining these subarrays after sorting each one of them. Your program should also find number of comparisons and inversions during sorting the array.

```
#include <iostream>
using namespace std;

int comparisonCount = 0;
int inversionCount = 0;

void merge(int a[], int start, int mid, int end) {

   int leftSize = mid - start + 1;
   int rightSize = end - mid;

   int left[50], right[50];

   for (int i = 0; i < leftSize; i++)
      left[i] = a[start + i];

   for (int j = 0; j < rightSize; j++)
      right[j] = a[mid + 1 + j];

   int i = 0, j = 0, k = start;

   while (i < leftSize && j < rightSize) {
      comparisonCount++;

      if (left[i] <= right[j]) {
         a[k] = left[i];
         i++;
      } else {
         a[k] = right[j];
         j++;
         inversionCount = inversionCount + (leftSize - i);
      }
      k++;
   }
```

```
    while (i < leftSize) {
        a[k] = left[i];
        i++;
        k++;
    }

    while (j < rightSize) {
        a[k] = right[j];
        j++;
        k++;
    }
}

void mergeSort(int a[], int start, int end) {

    if (start < end) {
        int mid = (start + end) / 2;

        mergeSort(a, start, mid);
        mergeSort(a, mid + 1, end);
        merge(a, start, mid, end);
    }
}

int main() {

    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int a[100];
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    mergeSort(a, 0, n - 1);
cout << "Sorted array:\n";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
 cout << "\nNumber of comparisons: " << comparisonCount;
    cout << "\nNumber of inversions: " << inversionCount;

    return 0;
}
```

OUTPUT:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-10.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
Enter number of elements: 10
Enter elements:
54 65 34 76 78 97 46 32 51 21
Sorted array:
21 32 34 46 51 54 65 76 78 97
Number of comparisons: 22
Number of inversions: 28
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

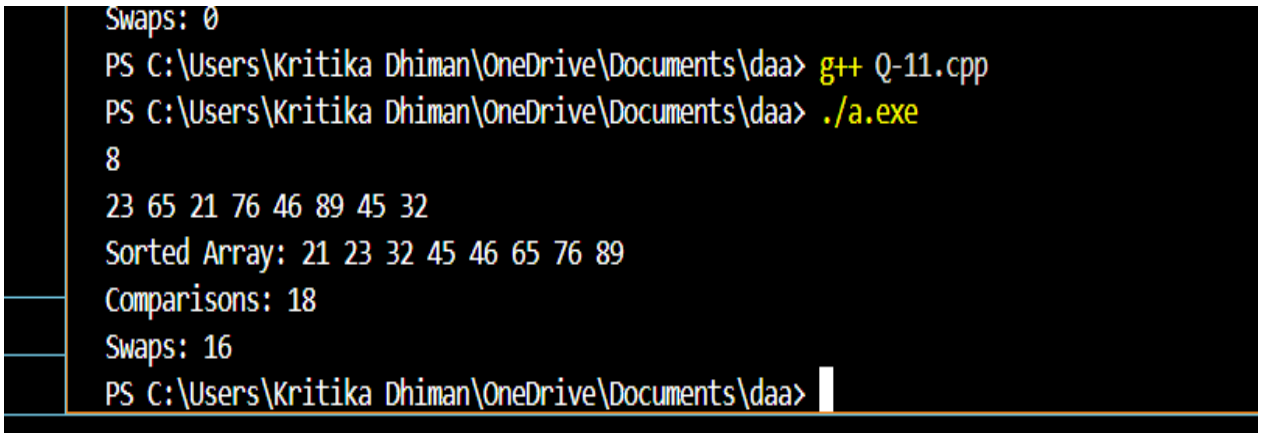
WEEK4:

II. Given an unsorted array of integers, design an algorithm and implement it using a program to sort an array of elements by partitioning the array into two subarrays based on a pivot element such that one of the sub array holds values smaller than the pivot element while another sub array holds values greater than the pivot element. Pivot element should be selected randomly from the array. Your program should also find number of comparisons and swaps required for sorting the array.

```
#include <iostream>
using namespace std;

int comparisons = 0;
int swaps = 0;

void swapValues(int &a, int &b) {
   int temp = a;
   a = b;
   b = temp;
   swaps++;
}

int partition(int arr[], int low, int high) {
   int pivot = arr[high];
   int i = low - 1;

   for (int j = low; j < high; j++) {
      comparisons++;
      if (arr[j] < pivot) {
         i++;
         swapValues(arr[i], arr[j]);
      }
   }

   swapValues(arr[i + 1], arr[high]);
   return i + 1;
}

void quickSort(int arr[], int low, int high) {
   if (low < high) {

      int mid = (low + high) / 2;
```

```
        swapValues(arr[mid], arr[high]);   // pivot moved to end

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    cin >> n;

    int arr[100];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    cout << "\nComparisons: " << comparisons;
    cout << "\nSwaps: " << swaps;

    return 0;
}
```

OUTPUT:

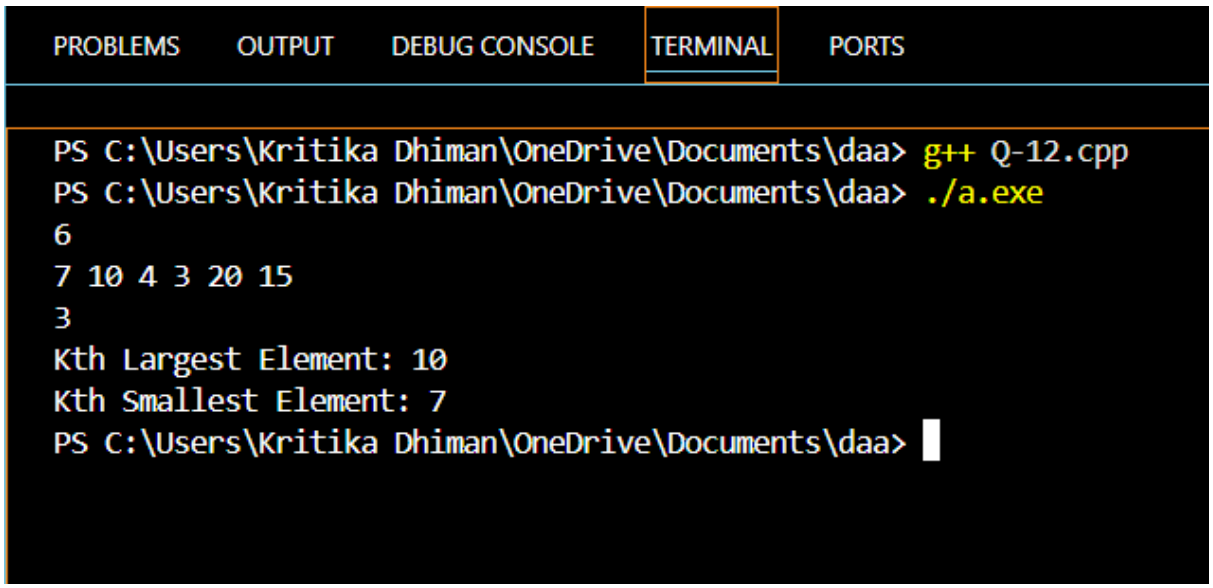

Swaps: 0
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-11.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
8
23 65 21 76 46 89 45 32
Sorted Array: 21 23 32 45 46 65 76 89
Comparisons: 18
Swaps: 16
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

WEEK4:

III.Given an unsorted array of integers, design an algorithm and implement it using a program to find Kth smallest or largest element in the array. (Worst case Time Complexity = O(n))

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    int n, k;
    cin >> n;

    int arr[100];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cin >> k;

    // Kth Largest (Min Heap)
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int i = 0; i < n; i++) {
        minHeap.push(arr[i]);
        if (minHeap.size() > k) {
            minHeap.pop();
        }
    }
    cout << "Kth Largest Element: " << minHeap.top() << endl;

    // Kth Smallest (Max Heap)
    priority_queue<int> maxHeap;
    for (int i = 0; i < n; i++) {
        maxHeap.push(arr[i]);
        if (maxHeap.size() > k) {
            maxHeap.pop();
        }
    }
    cout << "Kth Smallest Element: " << maxHeap.top() << endl;

    return 0;
}
```

OUTPUT:

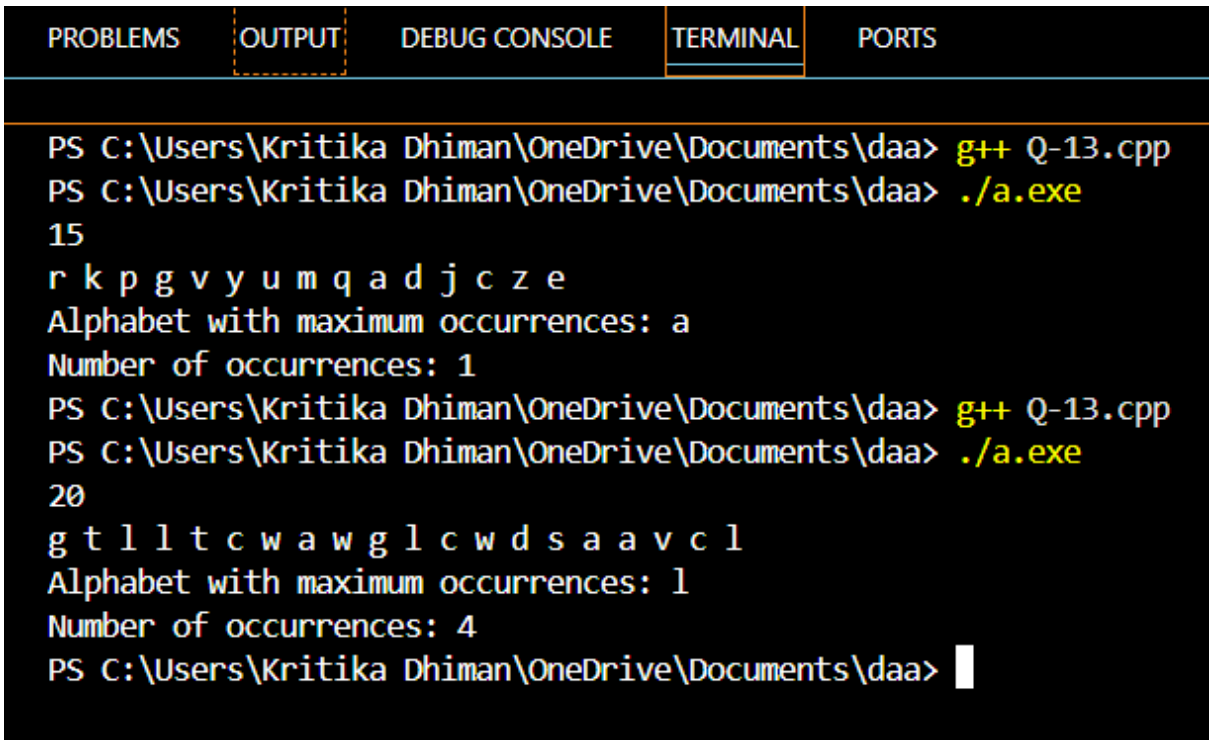

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-12.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
6
7 10 4 3 20 15
3
Kth Largest Element: 10
Kth Smallest Element: 7
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

WEEK5:

I. Given an unsorted array of alphabets containing duplicate elements. Design an algorithm and implement it using a program to find which alphabet has maximum number of occurrences and print it. (Time Complexity = O(n)) (Hint: Use counting sort)

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cin >> n;
   int count[26] = {0};
   char ch;

   for (int i = 0; i < n; i++) {
      cin >> ch;
      if (ch >= 'A' && ch <= 'Z') {
         ch = ch + 32;
      }
      else if (ch >= 'a' && ch <= 'z') {
         count[ch - 'a']++;
      }
   }

   int maxCount = count[0];
   char maxChar = 'a';

   for (int i = 1; i < 26; i++) {
      if (count[i] > maxCount) {
         maxCount = count[i];
         maxChar = char(i + 'a');
      }
   }

   cout << "Alphabet with maximum occurrences: " << maxChar << endl;
   cout << "Number of occurrences: " << maxCount << endl;

   return 0;
}
```

OUTPUT:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-13.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
15
r k p g v y u m q a d j c z e
Alphabet with maximum occurrences: a
Number of occurrences: 1
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-13.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
20
g t l l t c w a w g l c w d s a a v c l
Alphabet with maximum occurrences: l
Number of occurrences: 4
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

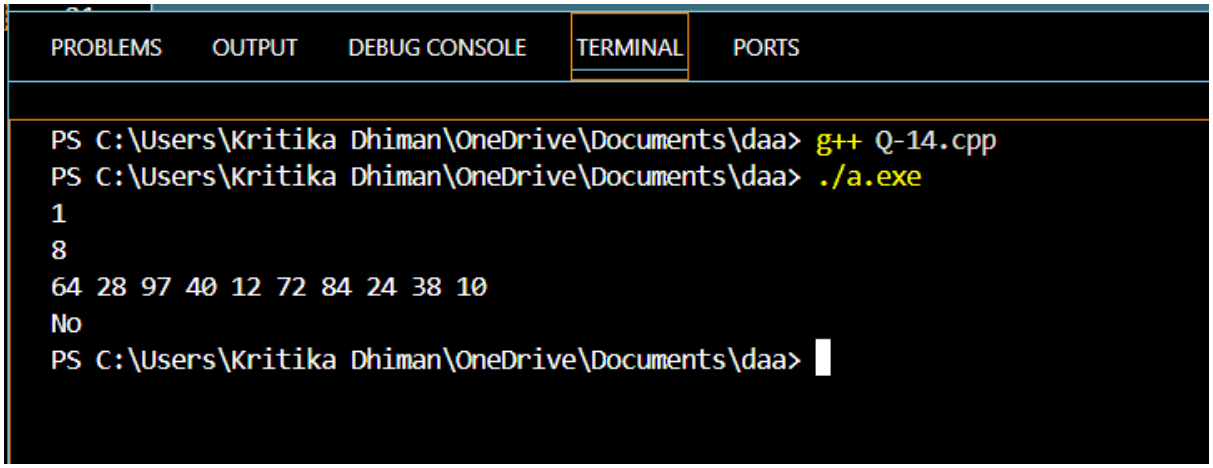
WEEK5:

II. Given an unsorted array of integers, design an algorithm and implement it using a program to find whether two elements exist such that their sum is equal to the given key element. (Time Complexity = O(n log n))

OUTPUT:

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
   int n;
   cin >> n;

   int arr[n];
   for(int i = 0; i < n; i++) {
      cin >> arr[i];
   } int key;
   cin >> key;

   sort(arr, arr + n);

   bool found = false;

   for(int i = 0; i < n - 1; i++) {
      int target = key - arr[i];
      if(binary_search(arr + i + 1, arr + n, target)) {
         found = true;
         break;
      }
   } if(found)
      cout << "Yes";
   else
      cout << "No";

   return 0;
}
```

OUTPUT:

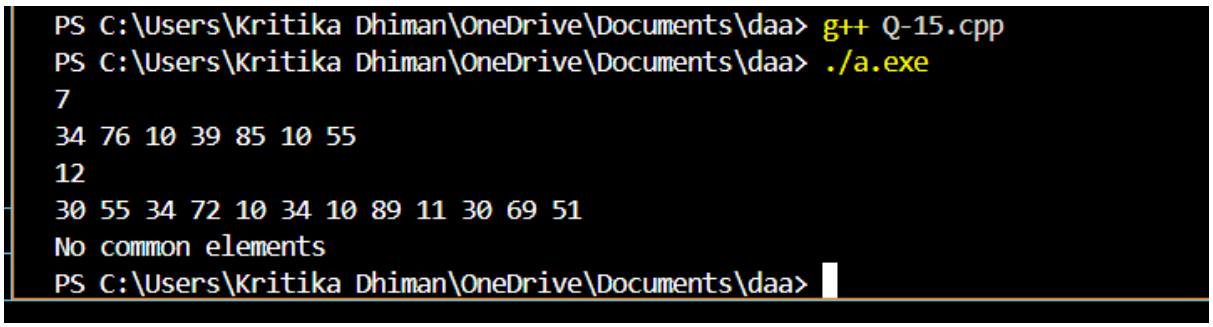

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-14.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
1
8
64 28 97 40 12 72 84 24 38 10
No
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

WEEK5:

III.You have been given two sorted integer arrays of size m and n. Design an algorithm and implement it using a program to find list
of elements which are common to both. (Time Complexity = O(m+n))

```
#include <iostream>
using namespace std;
int main() {
    int m, n;
    cin >> m;
   int a[m];
    for(int i = 0; i < m; i++) {
        cin >> a[i];
    }cin >> n;
 int b[n];
    for(int i = 0; i < n; i++) {
        cin >> b[i];
    }int i = 0, j = 0;
    bool found = false;
   while(i < m && j < n) {
        if(a[i] == b[j]) {
            cout << a[i] << " ";
            i++;
            j++;
            found = true;
        }
        else if(a[i] < b[j]) {
            i++;
        }
        else {
            j++;
        }
    }

    if(!found)
        cout << "No common elements";

    return 0;}
```

OUTPUT:


PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> g++ Q-15.cpp
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa> ./a.exe
7
34 76 10 39 85 10 55
12
30 55 34 72 10 34 10 89 11 30 69 51
No common elements
PS C:\Users\Kritika Dhiman\OneDrive\Documents\daa>

Design and Analysis of Algorithms

Name- KRITIKA DHIMAN Class Roll No. 32 University Roll No. 2418611 Section: DS1